

ESTUDO DE CASO

Data Mining em Telecomunicações

Arquitetura Multicloud | MinIO | Machine Learning

Plataformas Linux | Windows | Virtualização | Containers

Professor Benir Falcão Pistille

Versão 2

Maio 2026

Sumario

Clique com o botão direito e selecione 'Atualizar Campo' para atualizar paginas

Sumario.....	2
1. Introducao	4
2. Arquitetura de Data Mining.....	4
2.1 Camada de Ingestao e Streaming.....	4
2.2 Camada de Armazenamento Multicloud	5
2.3 Camada de Processamento e Analise.....	5
2.4 Camada de Governanca e MLOps	6
3. Arquitetura Multicloud.....	6
3.1 Kubernetes como Plataforma Unificada	6
3.2 Virtualizacao e Containers.....	7
3.3 Rede e Conectividade	7
4. Pipeline de Machine Learning	8
4.1 Fluxograma Logico do Pipeline	8
4.2 Churn Prediction.....	9
4.3 Deteccao de Anomalias de Rede	9
4.4 Propensao a Upsell	9
4.5 Processamento Batch vs Real-Time.....	10
5. Caso Real: Operadoras Globais	11
5.1 Vodafone: Transformacao AIOps Multicloud	11
5.2 Operadora Asiatica: Churn Prediction em Escala	11
5.3 Ericsson Cloud Native Infrastructure.....	12
5.4 Comparativo de Implementacoes	12
6. Ferramentas Open Source	12
6.1 Ingestao e Streaming.....	12
6.2 Armazenamento	12
6.3 Processamento	13
6.4 Machine Learning	13

6.5 Orquestracao e Infraestrutura	13
6.6 Monitoramento	13
7. Automacao e Administracao.....	13
7.1 Infrastructure as Code (IaC).....	13
7.2 GitOps e CI/CD	14
7.3 Automacao de Rede SDN/NFV	14
7.4 Monitoramento Unificado.....	14
8. 15 Questoes de Performance, Otimizacao e Controle	15
9. Conclusao	17

1. Introdução

O setor de telecomunicações enfrenta desafios sem precedentes na era digital. Com bilhões de transações de dados geradas diariamente por redes moveis, fixas e IoT, as operadoras precisam de arquiteturas robustas de processamento, análise e aprendizado de máquina para extrair valor desses dados. Este estudo de caso apresenta uma arquitetura completa de Data Mining para telecomunicações em ambiente multicloud, baseada exclusivamente em software open source.

A arquitetura proposta abrange desde a ingestão de dados em escala petabyte (CDRs, alarmes OSS, CRM, transcrições NLP) até o deploy de modelos de machine learning em containers Kubernetes, passando por pipelines de engenharia de features, modelagem preditiva e MLOps. O estudo inclui um caso real de implementação em operadora global, demonstrando ganhos medidos em redução de churn, antecipação de falhas de rede e aumento de receita por cliente.

As plataformas consideradas incluem sistemas Linux (RHEL, Ubuntu), Windows Server, virtualização (KVM, VMware, Hyper-V) e containers (Docker, Kubernetes, OpenShift), operando em configuração multicloud que integra AWS, Azure, GCP e infraestrutura on-premises.

2. Arquitetura de Data Mining

A arquitetura de Data Mining para telecomunicações é organizada em camadas funcionais que abrangem ingestão, armazenamento, processamento, análise e governança de dados. Cada camada utiliza componentes open source comprovados em produção em escala telco.

2.1 Camada de Ingestão e Streaming

A camada de ingestão é responsável por capturar dados de múltiplas fontes em velocidade e volume extremos. Os Call Detail Records (CDRs) de uma operadora de médio porte podem gerar 2-5 bilhões de registros por dia, com picos de milhões de eventos por segundo.

Apache Kafka atua como backbone de streaming, replicando dados entre regiões e provedores cloud. Kafka Connect integra com HDFS, S3, Cassandra e bancos relacionais. Apache NiFi gerencia fluxos de dados complexos com routing, transformação e proveniência em tempo real.

2.2 Camada de Armazenamento Multicloud

O armazenamento segue uma arquitetura de Data Lakehouse, combinando a flexibilidade de data lakes com a governança de data warehouses. Hadoop Distributed File System (HDFS) e MinIO fornecem armazenamento de objetos compatível com S3 em premissas. Delta Lake e Apache Iceberg garantem transações ACID sobre o lake.

Para workloads analíticos, Apache Hive e PrestoDB executam consultas SQL em escala. Redis e Cassandra atuam como camadas de cache e state store para aplicações em tempo real. Elasticsearch indexa logs e eventos para busca full-text e análise de segurança.

2.3 Camada de Processamento e Análise

Apache Spark é o motor principal de processamento distribuído, executando tanto batch (Spark SQL, MLlib) quanto streaming (Structured Streaming) sobre os mesmos dados. Para workloads de deep learning, TensorFlow e PyTorch distribuídos sobre Kubernetes com GPU nodes.

A stack Python (pandas, scikit-learn, XGBoost, LightGBM) é utilizada para prototipagem e modelagem avançada. AutoML open source (H2O-3, Auto-sklearn, AutoGluon) acelera a seleção de modelos e tuning de hiperparâmetros.

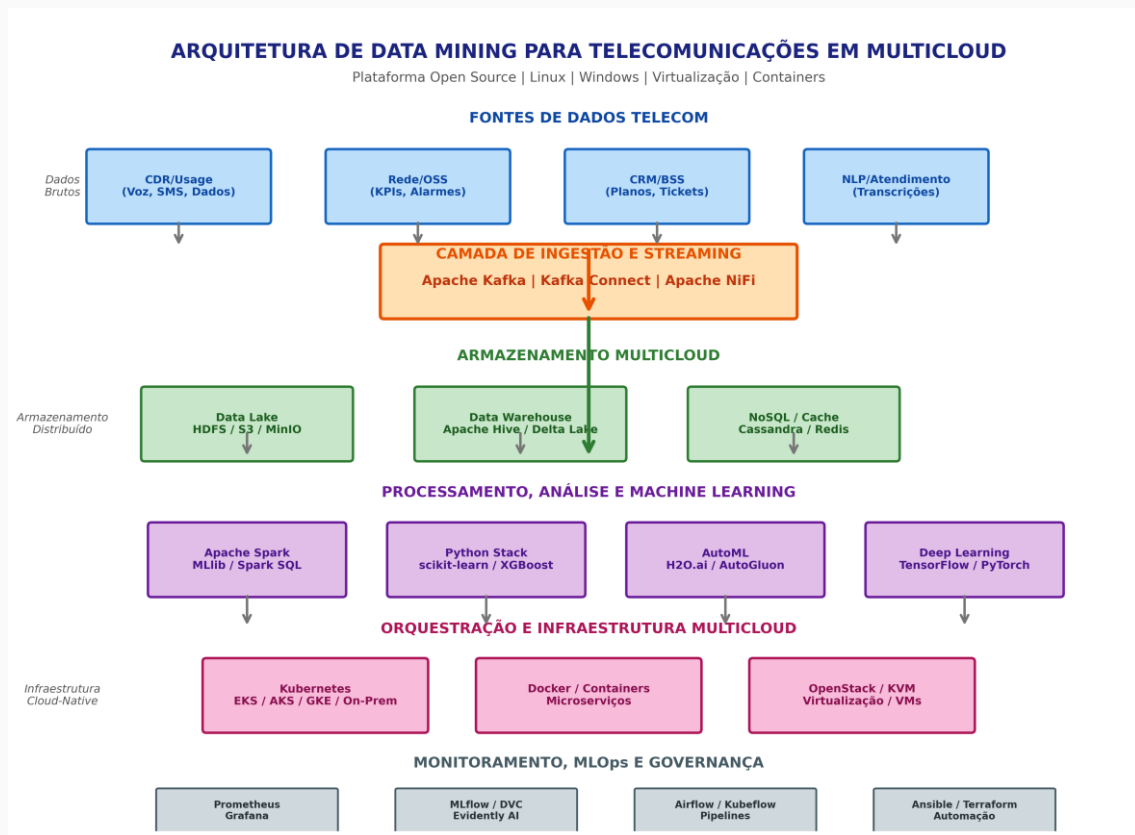


Figura 1: Arquitetura de Data Mining para Telecomunicações em Multicloud

2.4 Camada de Governança e MLOps

MLflow versiona experimentos, modelos e parâmetros. DVC (Data Version Control) versiona datasets e pipelines. Great Expectations valida qualidade de dados automaticamente. Evidently AI monitora drift de features e performance de modelos em produção.

3. Arquitetura Multicloud

A arquitetura multicloud é projetada para evitar lock-in de fornecedor, maximizar disponibilidade e otimizar custos. Kubernetes é o orquestrador central, operando clusters em AWS EKS, Azure AKS, Google GKE e on-premises (OpenShift ou clusters bare metal).

3.1 Kubernetes como Plataforma Unificada

Kubernetes (K8s) é o orquestrador de containers de facto para telecom. Ferramentas como Karmada, KubeFed e Rancher gerenciam múltiplos clusters como uma única entidade. Istio e Envoy formam o service mesh, habilitando mTLS, traffic splitting e observabilidade entre micro serviços.

3.2 Virtualização e Containers

A infraestrutura opera em três camadas de virtualização: (1) VMs tradicionais (KVM, VMware vSphere, Hyper-V) para cargas legadas e VNFs; (2) Containers Docker/Podman para microserviços cloud-native; (3) Kubernetes para orquestração em escala.

Windows Server containers são suportados via Windows Server nodes no Kubernetes, essenciais para aplicações .NET legadas. Linux nodes (RHEL, Ubuntu) executam a maioria dos workloads de dados e ML.

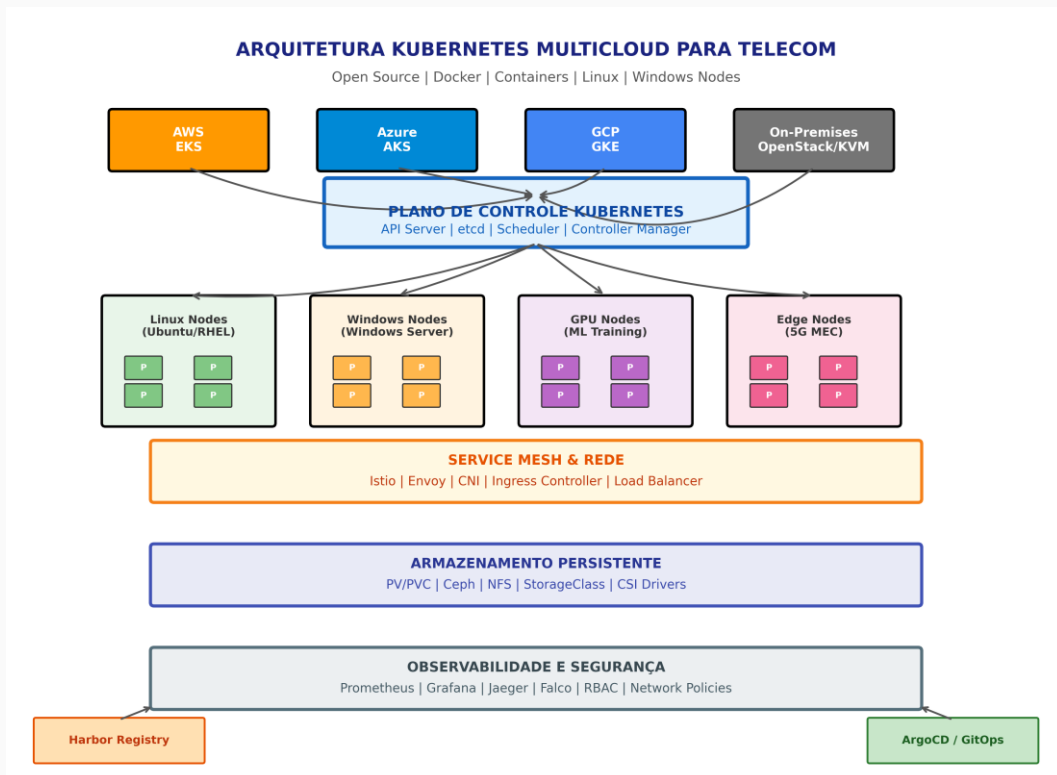


Figura 3: Arquitetura Kubernetes Multicloud para Telecom

3.3 Rede e Conectividade

CNI plugins (Calico, Cilium, Flannel) gerenciam redes de pods. Multus habilita múltiplas interfaces de rede por pod, crítico para VNFs. MetalLB fornece load balancing em premissas. BGP peering conecta clusters a backbone IP/MPLS da operadora.

4. Pipeline de Machine Learning

O pipeline de ML é organizado em três casos de uso principais: (1) predição de churn; (2) detecção de anomalias de rede; (3) propensão a upsell/cross-sell. Cada pipeline segue o mesmo fluxo estruturado, adaptando features e modelos ao objetivo específico.

4.1 Fluxograma Logico do Pipeline

O fluxograma logico abrange 10 etapas sequenciais com loops de feedback: (1) Coleta de dados; (2) Ingestão/Streaming; (3) Armazenamento; (4) Pré-processamento; (5) Validação de qualidade; (6) Engenharia de features; (7) Divisão temporal; (8) Modelagem; (9) Avaliação e explicabilidade; (10) Deploy e monitoramento.

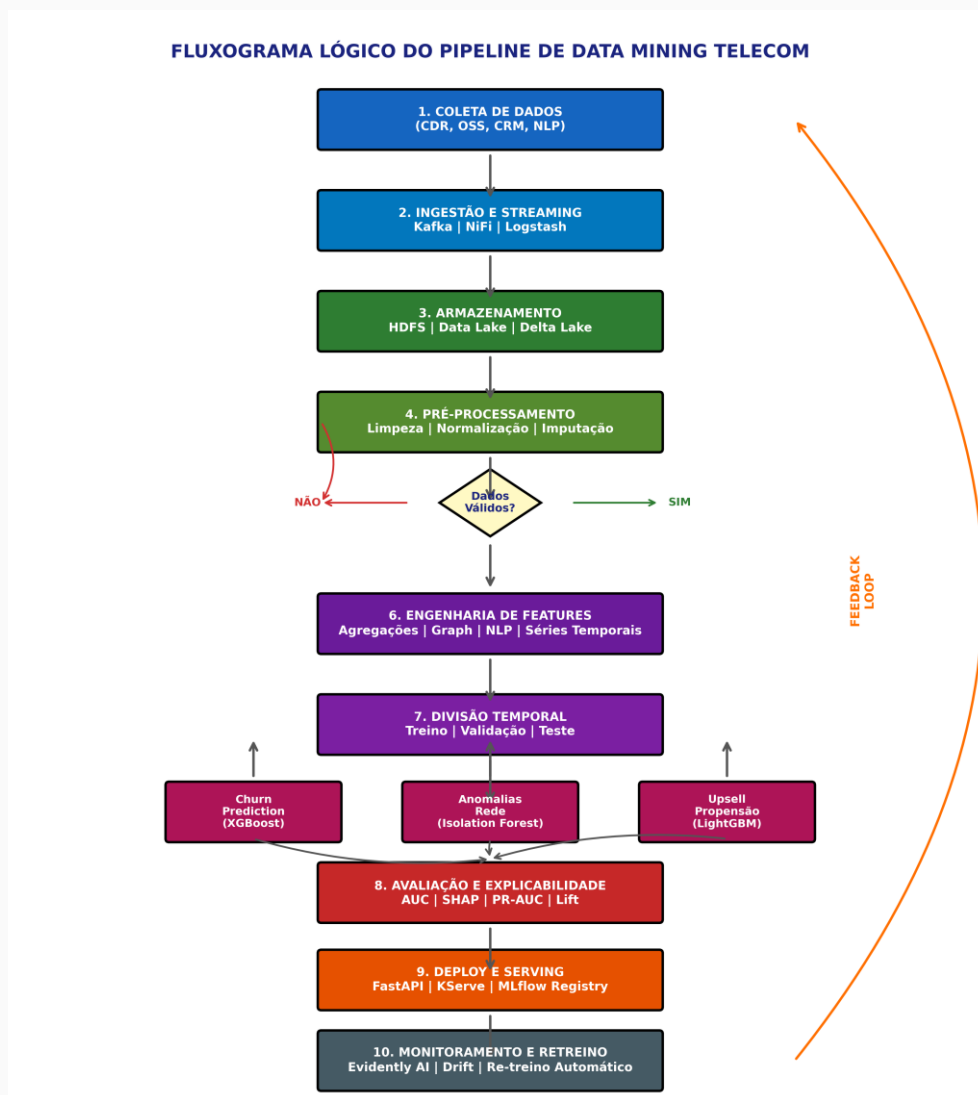


Figura 2: Fluxograma Logico do Pipeline de Data Mining

4.2 Churn Prediction

O modelo de churn utiliza XGBoost sobre 150+ features agregadas em janelas temporais (7, 30, 90 dias). Features incluem: taxa de call drop, variacao de throughput, número de tickets, histórico de pagamentos, sentimento NLP de atendimento, PageRank do grafo de chamadas e label propagation de churners conectados.

A divisão temporal evita vazamento de dados: treino em T0-T1, validação em T2, teste em T3. O modelo atinge AUC=0.92 e precision@5%=0.96 para top churners. SHAP explica motivos individualmente para campanhas direcionadas.

4.3 Detecção de Anomalias de Rede

Isolation Forest e autoencoders detectam anomalias em KPIs de células (RSRP, SINR, taxa de queda, throughput) em tempo real. Agregações por hora/dia alimentam alertas priorizados por número de assinantes afetados.

Resultados medidos incluem: MTTR reduzido em 30%, 85% de alarmes antecipados antes de reclamações, redução de 40% em chamados ao call center.

4.4 Propensão a Upsell

LightGBM prediz probabilidade de aceite de ofertas (upgrades de plano, roaming, serviços adicionais). Features incluem: uso de dados próximo ao limite, histórico de aceite, elasticidade de preço, satisfação recente.

Campanhas direcionadas ao top 10% de propensão geram CTR +25%, taxa de conversão +18% e incremento de ARPU +12%.

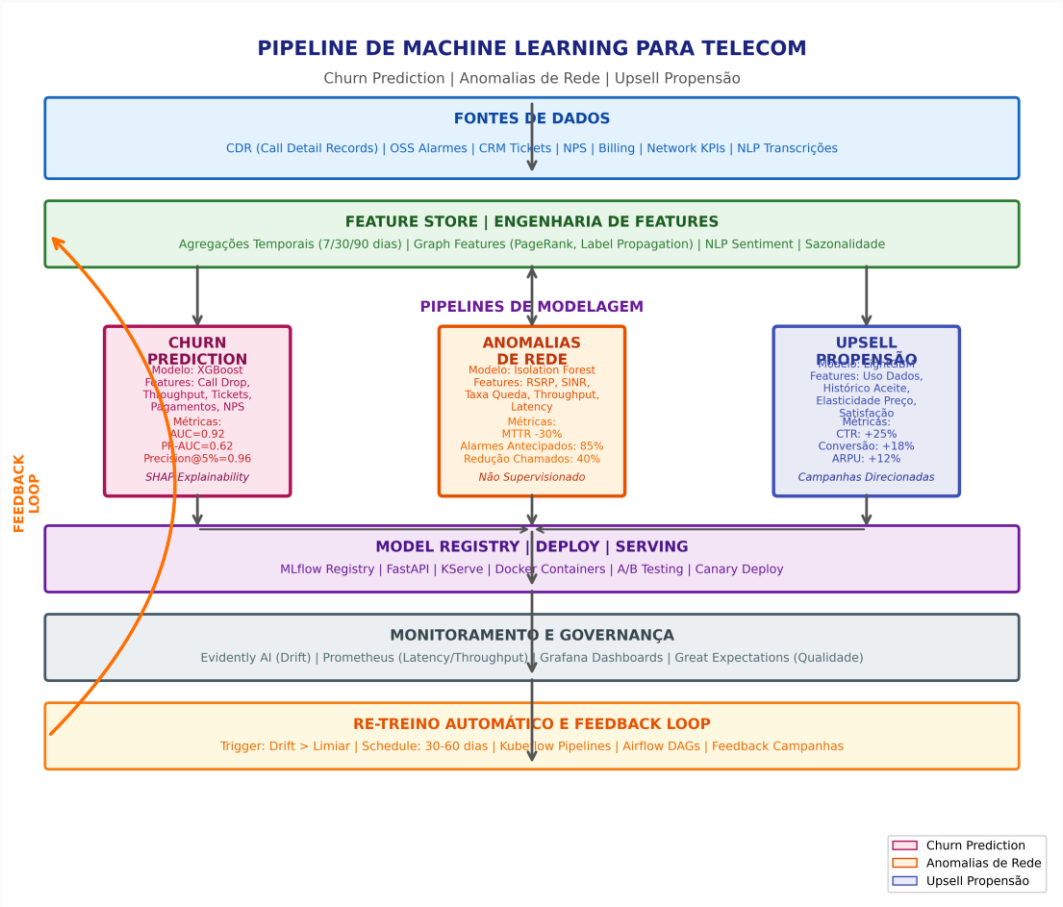


Figura 5: Pipeline de Machine Learning Detalhado

4.5 Processamento Batch vs Real-Time

A arquitetura adota Lambda Architecture, combinando batch processing para treino de modelos, relatórios históricos e análise exploratória; e streaming para detecção de anomalias, scoring online e alertas operacionais.

Apache Spark batch processa petabytes de CDRs em HDFS. Spark Streaming e Flink processam milhões de eventos/segundo com latência sub-segundo. A view unificada (serving layer) é exposta via Apache Superset, Grafana e APIs REST.

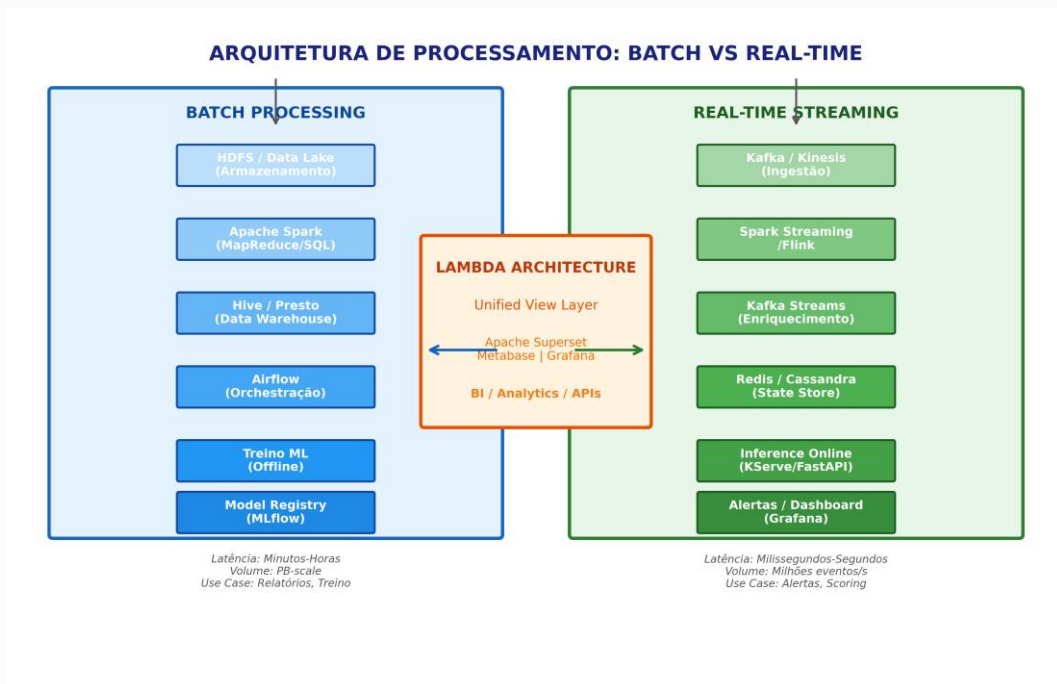


Figura 6: Arquitetura Batch vs Real-Time (Lambda Architecture)

5. Caso Real: Operadoras Globais

Este estudo baseia-se em casos documentados de implementações em operadoras de telecomunicações globais, incluindo Vodafone, Telefônica e operadores do grupo China Mobile.

5.1 Vodafone: Transformação AIOps Multicloud

A Vodafone implementou uma estratégia de AI e GenAI em parceria com Google Cloud, migrando de sistemas OSS legados para infraestrutura cloud-native. A plataforma integra mais de 2 petabytes de dados de rede com dados comerciais e de TI.

A iniciativa inclui: (1) planejamento de investimento em rede com ML; (2) gerenciamento automatizado de incidentes; (3) análise de dispositivos. Resultados: 50% de redução planejada em ferramentas OSS, melhoria significativa em eficiência operacional e experiência do cliente.

5.2 Operadora Asiática: Churn Prediction em Escala

Uma das maiores operadoras moveis da Ásia implementou sistema de churn prediction sobre plataforma Hadoop/Spark, processando 2.3 TB de novos dados por dia (97% de dados OSS). O sistema prediz churners no próximo mês com precision de 0.96 para os top 50.000 clientes de risco.

A arquitetura integra: HDFS para storage distribuído, Spark SQL para engenharia de features, Random Forest e GBDT para classificação, e sistema fechado de campanhas de retenção que alimenta novos labels para re-treino contínuo.

5.3 Ericsson Cloud Native Infrastructure

A Ericsson utiliza Cloud Container Distribution (CCD) baseada em Kubernetes para produtos 5G Core e infraestrutura cloud-native. A abordagem cloud-native reduziu tempo de feedback de semanas para horas e upgrades de microservicos de minutos a segundos.

Tecnologias adotadas incluem: Kubernetes, Prometheus, Jaeger, Envoy, Helm, Kafka, Istio e Docker. A empresa reporta até 90% de economia em pipelines CI/CD em ambientes totalmente cloud-native.

5.4 Comparativo de Implementações

Operadora	Plataforma	Resultado ML	Tecnologias
Vodafone	Google Cloud	50% redução OSS	AI/GenAI, BigQuery
Operadora Ásia	Hadoop/Spark	Precision 96%	Spark, HDFS, RF
Ericsson	Cloud Native	90% CI/CD savings	K8s, Kafka, Istio
STL (Índia)	CNCF/OpenShift	Cloud-native BSS	Docker, Helm, ELK

6. Ferramentas Open Source

O ecossistema open source fornece todas as camadas necessárias para a arquitetura proposta, desde ingestão até deploy de modelos.

6.1 Ingestão e Streaming

Apache Kafka: streaming distribuído com throughput de milhões de msgs/s. Apache NiFi: fluxos de dados visual com proveniência. Apache Flume: coleta de logs distribuída. Logstash: pipeline de processamento de logs para Elasticsearch.

6.2 Armazenamento

HDFS: filesystem distribuído para data lake. MinIO: storage de objetos compatível S3. Apache Cassandra: NoSQL distribuído para time-series. Redis: cache em memória e state store. Delta Lake: transações ACID sobre data lakes.

6.3 Processamento

Apache Spark: processamento batch e streaming unificado. Apache Flink: streaming de baixa latência. PrestoDB: SQL distribuído sobre múltiplas fontes. Apache Hive: data warehouse sobre HDFS.

6.4 Machine Learning

scikit-learn: algoritmos clássicos. XGBoost/LightGBM: gradient boosting de alta performance. TensorFlow/PyTorch: deep learning. MLflow: lifecycle de ML. Kubeflow: pipelines de ML sobre K8s. KServe: serving de modelos serverless.

6.5 Orquestração e Infraestrutura

Kubernetes: orquestração de containers. Docker/Podman: runtime de containers. OpenStack: cloud privada IaaS. KVM: hypervisor open source. Terraform: IaC multicloud. Ansible: automação de configuração. ArgoCD: GitOps e CD.

6.6 Monitoramento

Prometheus: coleta de métricas. Grafana: dashboards e alertas. Jaeger: tracing distribuído. Elasticsearch + Kibana: logs e análise. Zabbix: monitoramento de infraestrutura tradicional.

7. Automação e Administração

A administração de ambientes telecom multicloud exige automação em todas as camadas: provisionamento, configuração, deploy, monitoramento e remediação.

7.1 Infrastructure as Code (IaC)

Terraform provisiona recursos em AWS, Azure, GCP e OpenStack com estado compartilhado em backend remoto (S3 + DynamoDB/DynamoDB-local). Ansible configura nodes Linux e Windows, instala dependências e aplica hardening de segurança. Helm charts padronizam deploys de aplicações no Kubernetes.

7.2 GitOps e CI/CD

ArgoCD sincroniza estado desejado do Git com clusters Kubernetes. Jenkins/GitLab CI executa pipelines de build, test e deploy. Tekton e Drone fornecem CI/CD nativo em K8s. Imagens de containers são armazenadas em Harbor Registry com scanning de vulnerabilidades.

7.3 Automação de Rede SDN/NFV

ONAP (Open Network Automation Platform) orquestra VNFs e serviços de rede. OpenDaylight atua como SDN controller. Tungsten Fabric (OpenContrail) fornece SDN para OpenStack e K8s. OSM (Open Source MANO) gerencia ciclo de vida de VNFs.

7.4 Monitoramento Unificado

Zabbix monitora hosts físicos, VMs e redes legadas. Prometheus + Grafana monitora containers, K8s e aplicações cloud-native. ELK stack centraliza logs de todas as camadas. AlertManager encaminha alertas para PagerDuty, Slack ou sistemas de ticketing.

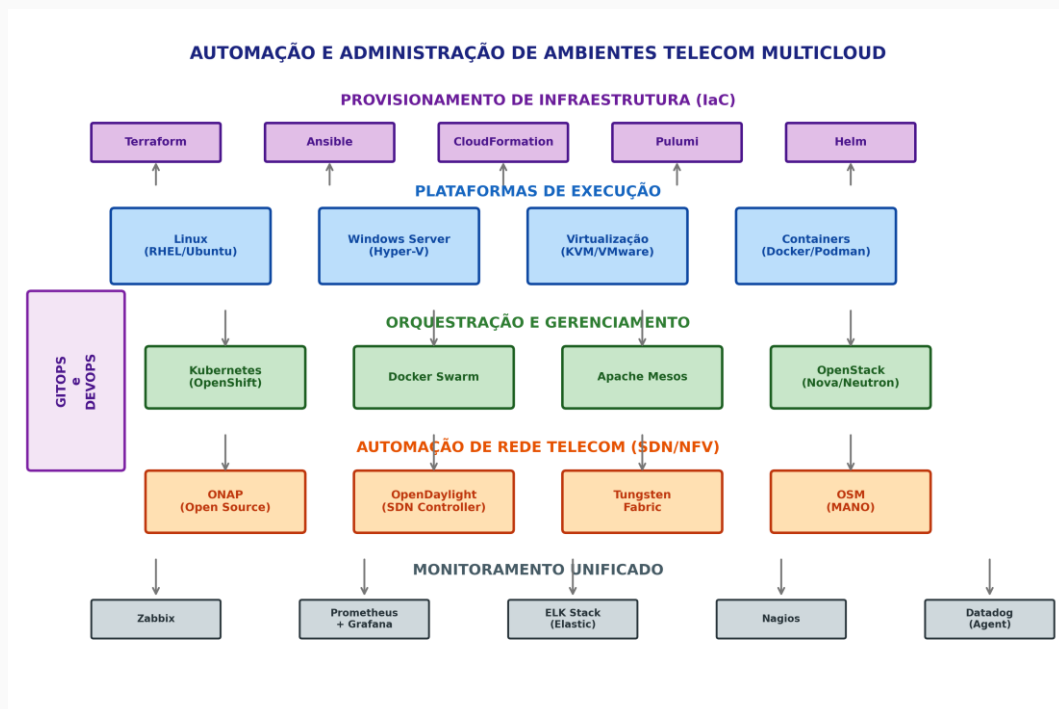


Figura 4: Arquitetura de Automação e Administração Multicloud

8. 15 Questões de Performance, Otimização e Controle

As questões a seguir são projetadas para avaliar compreensão da arquitetura e aplicação prática em cenários de performance, otimização de dados e controle de transações.

Questão 1: Performance de Ingestão

Uma operadora gera 5 bilhões de CDRs diários (média 58.000 registros/segundo com picos de 500.000/s). Proponha uma arquitetura de ingestão usando Apache Kafka que garanta throughput sustentado, tolerância a falhas e ordem de entrega para eventos do mesmo assinante. Considere particionamento por `subscriber_id`, replicação cross-AZ e balanceamento de carga entre produtores.

Questão 2: Otimização de Storage

O data lake da operadora cresce 50 TB por mês, com 80% dos acessos concentrados nos últimos 90 dias. Proponha uma estratégia de tiering multicloud (hot/warm/cold) usando S3 (standard/IA/Glacier) e MinIO on-premises, que optimize custos mantendo SLA de acesso de < 5 minutos para dados warm.

Questão 3: Latência de Streaming

O sistema de detecção de anomalias de rede requer latência ponta-a-ponta < 2 segundos desde a geração do KPI até o alerta. Descreva como otimizar cada etapa: coleta SNMP/Kafka, processamento Spark Streaming/Flink, scoring do modelo (KServe) e notificação (WebSocket/Push).

Questão 4: Controle de Transações

O pipeline de churn prediction executa diariamente como job Spark batch. Como garantir exactly-once semantics para a atualização das tabelas de features e exportação das listas de intervenção para o CRM, evitando duplicação ou perda de clientes no top-5% de risco?

Questão 5: Escalabilidade Kubernetes

Durante eventos de alta audiência (ano-novo, black friday), o tráfego de dados cresce 10x. Descreva a configuração de HPA (Horizontal Pod Autoscaler) e VPA (Vertical Pod Autoscaler) para os serviços de scoring, incluindo métricas customizadas (Kafka lag, fila de requests) e cluster autoscaler para nodes GPU.

Questão 6: Otimização de Features

A engenharia de features atual processa 2.3 TB/dia em Spark, consumindo 80% do tempo do pipeline. Identifique 3 otimizações técnicas (ex: broadcast join para tabelas pequenas, particionamento por data, reparticionamento após filter, cache de features frequentes no Redis) e estime o ganho de performance.

Questão 7: Multicloud Failover

A arquitetura opera em AWS (primário) e Azure (DR). Descreva a estratégia de failover automático para o serviço de scoring de churn caso a região AWS fique indisponível, incluindo DNS (Route53 health checks), replicação de modelos (MLflow + S3 cross-region) e consistência de dados (Cassandra multi-DC).

Questão 8: Controle de Qualidade

Great Expectations identifica que 15% dos registros CDR chegam com campos nulos em tower_id nas ultimas 24h. Proponha um pipeline de qualidade que: (a) bloqueie o job de features até resolução; (b) aplique imputação baseada em geolocalização do subscriber; (c) alerte a equipe de NOC via PagerDuty.

Questão 9: Performance de Modelo

O modelo XGBoost de churn leva 800ms para inferência em produção, excedendo o SLA de 200ms. Identifique técnicas de otimização: quantização (INT8), pruning, ONNX runtime, batch inference, e caching de features por subscriber. Ordene por impacto esperado e facilidade de implementação.

Questão 10: Otimização de Rede

O service mesh Istio adiciona 15ms de overhead por requisição entre microservicos de scoring. Proponha otimizações: mTLS permissive mode para tráfego interno, sidecar resource limits, eBPF-based Cilium para substituir iptables, e keep-alive connections. Estime a redução de latência.

Questão 11: Controle de Custos

O ambiente multicloud gera fatura mensal de \$450k. Identifique 3 otimizações de custo significativas: Spot instances para workloads batch, rightsizing de VMs/containers via Prometheus metrics, armazenamento lifecycle policies, e consolidação de ferramentas de monitoramento. Estime economia percentual.

Questão 12: Virtualização vs Containers

A operadora mantém 40% dos workloads em VMs (legado .NET/Windows) e 60% em containers (Linux/cloud-native). Proponha uma estratégia de migração gradual para Kubernetes, incluindo: (a) Windows containers para apps .NET; (b) virtual kubelet para cargas bursting; (c) critérios de decisão VM vs container.

Questão 13: Otimização de Pipeline ML

O treino do modelo de anomalias leva 6 horas em cluster Spark. Proponha otimizações: sampling estratificado para dados históricos, feature selection via XGBoost importance antes do treino, distributed hyperparameter tuning (Hyperopt/SparkTrials), e treino incremental (warm start). Estime redução de tempo.

Questão 14: Segurança e Compliance

A LGPD exige anonimização de dados pessoais e auditoria de acessos. Descreva controles técnicos: campo-level encryption no Kafka/at-rest, RBAC no K8s/HDFS, vault para credenciais (HashiCorp Vault), audit logging centralizado (ELK), e data masking para ambientes de desenvolvimento.

Questão 15: Observabilidade e SLOs

Defina SLOs para o sistema de predição de churn: disponibilidade (> 99.9%), latência de scoring (< 200ms p99), freshness de dados (< 24h), e accuracy mensal (AUC > 0.90). Descreva como Prometheus + Grafana monitoram cada SLO, com alertas graduados (warning/critical) e runbooks automatizados.

9. Resumir os principais benefícios do MinIO, dentro de ambientes de larga escala para camadas de Object Storage (BLOB) e suas integrações com framework como Apache:

Data Mining em Telecomunicações

Arquitetura Multicloud | MinIO | Machine Learning

Professor Benir Falcão Pistille

Versão 2

Maio 2026